Отчет второго блока

ФИО: Михайлов Петр Сергеевич
ИСУ: 466730
Группа: P3211

1 Коровы в стойла

1.1 Код задачи

```
#include <iostream>

using namespace std;

long long q, w, e, r, t, y, u, i, o, p, a[100005], s, d;

int main() {
  if (cin >> q >> w) {
    e = 0;
    while (e < q) {
      cin >> a[e];
      e = e + 1;
    }
    r = 0;
    t = 1000000009;
    y = 0;
    while (r <= t) {
      u = r + t;
      u = u / 2;
      i = 1;
      o = a[0];
      p = 1;
      while (p < q) {
        s = a[p];
        d = s - o;
        if (d >= u) {
          i = i + 1;
          o = a[p];
        } else {
          d = 0;
        }
        p = p + 1;
      }
      if (i == w || i > w) {
        y = u;
        r = u + 1;
      } else {
        t = u - 1;
      }
    }
    cout << y << "\n";
  }
  return 0;
}
```

1.2 Этапы мышления

Основная идея: Использование бинарного поиска по ответу для нахождения максимально возможного минимального расстояния между ковбоями.

Алгоритм:

- Диапазон поиска задается от 0 до максимально возможной координаты (10^9).
- Для каждого среднего значения (`u`) проверяется, можно ли расставить `w` ковбоев так, чтобы расстояние между любыми двумя было не меньше `u`. Проверка осуществляется

жадным способом: ставим первого ковбоя в начало, а каждого следующего — в первое доступное стойло на расстоянии $\geq u$.

- Если расстановка возможна, пытаемся увеличить расстояние (`r = u + 1`), если нет — уменьшаем (`t = u - 1`).

2 Число

2.1 Код задачи

```cpp
#include <cstdio>
#include <iostream>
#include <string>

using namespace std;

string a[105], z, x, c, v, b;
long long q, w, e, r, t;

int main() {
  freopen("number.in", "r", stdin);
  freopen("number.out", "w", stdout);

  q = 0;
  while (cin >> z) {
    a[q] = z;
    q = q + 1;
  }

  w = 0;
  while (w < q) {
    e = 0;
    t = q - 1;
    while (e < t) {
      x = a[e];
      c = a[e + 1];
      v = x + c;
      b = c + x;

      if (v < b) {
        a[e] = c;
        a[e + 1] = x;
      } else {
        r = 0;
      }

      if (r == 0) {
        e = e + 1;
      }
    }
    w = w + 1;
  }

  w = 0;
  while (w < q) {
    cout << a[w];
    w = w + 1;
  }

  cout << "\n";
  return 0;
}
```

2.2 Этапы мышления

Основная идея: Жадная сортировка строк на основе сравнения конкатенаций. Стандартная лексикографическая сортировка не подходит (например, «10» и «1»).

Алгоритм:

- Считываем все числа как строки.
- Используем алгоритм сортировки (пузырек), где критерием сравнения двух строк `x` и `c` является результат их сложения: если `x + c < c + x`, то строки нужно поменять местами.
- В результате самые «выгодные» для начала числа перемещаются в начало массива. Последовательный вывод дает максимально возможное число.

3 Кошмар в замке

3.1 Код задачи

```
#include <cstdio>
#include <iostream>
#include <string>

using namespace std;

string z;
long long a[35], s[35], v[35], d, f, g, h, j, k, l, q, w, e, r, t, y, u, i;
char o[100005];

int main() {
  freopen("aurora.in", "r", stdin);
  freopen("aurora.out", "w", stdout);

  if (cin >> z) {
    q = 0;
    while (q < 26) {
      cin >> a[q];
      s[q] = 0;
      v[q] = 0;
      q = q + 1;
    }

    w = 0;
    e = z.length();
    while (w < e) {
      r = z[w] - 97;
      s[r] = s[r] + 1;
      w = w + 1;
    }

    t = 0;
    y = e - 1;
    u = 0;

    while (u == 0) {
      i = -1;
      j = -1;
      k = 0;
      while (k < 26) {
        if (s[k] > 1) {
          if (v[k] == 0) {
            if (a[k] > j) {
              j = a[k];
              i = k;
            }
          }
        }
        k = k + 1;
      }

      if (i > -1) {
        o[t] = i + 97;
        o[y] = i + 97;
```

```
        t = t + 1;
        y = y - 1;
        s[i] = s[i] - 2;
        v[i] = 1;
      } else {
        u = 1;
      }
    }

    l = 0;
    while (l < 26) {
      f = 0;
      while (f < s[l]) {
        if (t <= y) {
          o[t] = l + 97;
          t = t + 1;
        }
        f = f + 1;
      }
      l = l + 1;
    }

    g = 0;
    while (g < e) {
      cout << o[g];
      g = g + 1;
    }
    cout << "\n";
  }

  return 0;
}
```

3.2 Этапы мышления

Основная идея: Построение палиндрома с максимальным «весом». Вес определяется стоимостью символов, стоящих на одинаковых позициях от краев.

Алгоритм:

- Считаем частоту каждого символа и сохраняем его стоимость.
- Жадным образом выбираем доступный символ с максимальной стоимостью, если его количество ≥ 2. Ставим его по краям результирующей строки.
- После того как дорогие пары закончились, заполняем оставшуюся середину строки всеми неиспользованными символами в алфавитном порядке.

4 Магазин

4.1 Код задачи

```
#include <iostream>

using namespace std;

long long q, w, e, r, t, y, u, i, o, p, a[10005], s, d, f;

int main() {
  if (cin >> q >> w) {
    e = 0;
    while (e < 10005) {
      a[e] = 0;
      e = e + 1;
    }

    r = 0;
```

```
    while (r < q) {
      cin >> t;
      a[t] = a[t] + 1;
      r = r + 1;
    }

    y = 0;
    u = 10000;
    i = 1;

    while (u > 0) {
      o = a[u];
      while (o > 0) {
        if (i == w) {
          i = 1;
          p = 0;
        } else {
          y = y + u;
          i = i + 1;
          s = 1;
        }
        o = o - 1;
        if (s == 1) {
          d = 0;
        }
      }
      u = u - 1;
    }

    cout << y << "\n";
  }
  return 0;
}
```

4.2 Этапы мышления

Основная идея: Минимизация общей стоимости покупок в магазине за счет жадного использования акции «каждый k-й товар бесплатно».

Алгоритм:

- Чтобы сэкономить больше всего, нужно сделать бесплатными самые дорогие товары из возможных.
- Для этого все цены сортируются в порядке убывания. В данном коде используется сортировка подсчетом (`array a[10005]`), так как диапазон цен ограничен.
- Мы идем по отсортированным ценам от самых больших к меньшим. Каждый w-й товар мы пропускаем (не добавляем к итоговой сумме `y`), тем самым делая его бесплатным по условию акции.